

NATIONAL UNIVERSITY OF COMPUTER & EMERGING SCIENCES ISLAMABAD
CAMPUS

CS-1002 Programming Fundamentals Fall-2023

ASSIGNMENT-02

Section (A, B, C, D, E, F, G, H, J and K)

Deadline: 20th October 2022 03:00 PM.

Instructions:

- For each question in your assignment, make a separate cpp file e.g. for question 1, make ROLL-
NUM_SECTION_Q#.cpp (23i-0001_A_Q1.cpp) and so on. Each file that you submit must contain
your name, student-id, and assignment # on top of the file in comments.
- Combine all your work in one folder. The folder must contain only .cpp files (no binaries, no exe
files etc.).
- Rename the folder as ROLL-NUM_SECTION (e.g. 23i-0001_A) and compress the folder as a zip file.
(e.g. 23i-0001_A.zip). Do not submit .rar file.
- All the submissions will be done on Google classroom within the deadline. Submissions other than
Google classroom (e.g. email etc.) will not be accepted.
- The student is solely responsible to check the final zip files for issues like corrupt files, viruses in the
file, mistakenly exe sent. If we cannot download the file from Google classroom due to any reason,
it will lead to zero marks in the assignment.
- Displayed output should be well mannered and well presented. Use appropriate comments and
indentation in your source code.
- **Be prepared for viva that involves modifying code after the submission of assignment.**
- If there is a syntax error in code, zero marks will be awarded in that part of assignment.
- Understanding the assignment is also part of assignment.
- **The AIM of this assignment is to give you practice with loops and if else statements.**
- **Zero marks will be awarded to the students involved in plagiarism. (Copying from the internet is
the easiest way to get caught).**
- **Late Submission policy will be applied as described in course outline.**

Tip: For timely completion of the assignment, start as early as possible.

Note: Follow the given instruction to the letter, failing to do so will result in a zero.

Question 1

Once upon a time in the digital kingdom, there was a wise sorcerer named Binaryus who had a peculiar talent for encoding numbers into the language of machines: binary. One day, a curious traveler named Numericus approached Binaryus with a desire to encode a non-negative integer into binary format.

Numericus presented a number (user input) to Binaryus, a rather sizable integer in the land of numerals. The sorcerer decided to embark on a journey to break down this number into its binary form, which consists of only zeros and ones.

As they began this mystical endeavor, Binaryus envisioned a grand castle representing the number. This castle had many gates, each holding a unique part of the number's binary secrets. Binaryus determined to organize these gates in a specific fashion.

"I shall separate the bits into four gates," Binaryus declared. "Three gates shall bear the burden of ten bits each, and the final gate shall be adorned with two bits."

With determination in his eyes, Binaryus divided the number into its binary representation. Each digit was meticulously converted to binary, and the gates were assigned their appropriate bits. In the first gate, labeled Gate 1, 10 bits were carefully arranged, representing a portion of the binary code for the number. It was as if the sorcerer placed a sentinel to guard this gate and ensure its security. In the second gate, labeled Gate 2, another 10 bits stood guard. These bits represented the second part of the number's binary tale, adding to the mystique of this magical encoding process. The third gate, aptly named Gate 3, held the last set of 10 bits, completing the trio of binary revelations. It was evident that Binaryus had a keen eye for organizing these bits into neat groups. Lastly, in the final gate, labeled Gate 4, two bits stood as the sentinel's companion. These bits represented the remaining, modest portion of the binary representation, concluding the encoding process.

With the four gates standing proudly in their formation, Binaryus invited Numericus to witness the culmination of their labor. Numericus marveled at the spectacle, appreciating the precision and magic that went into encoding the number into the language of machines. And thus, in the kingdom of Binaryus, the number found its place in the grand scheme of the binary world, forever etched into the gates as a testament to the artistry of sorcery and numbers.

In the realm of binary enchantment, Binaryus had designed Gate 1 to serve as a versatile decision-making input, allowing travelers like Numericus to wield its power for transformative operations. Each operation was like a unique charm, waiting to be cast upon the encoded number.

Binaryus explained the nature of Gate 1's magical abilities to Numericus, who was eager to utilize this newfound knowledge.

- "The first operation," Binaryus began, "is to convert the encoded input back to its original numerical form. Let us cast the spell of conversion."
 - With a flick of Binaryus's sorcerer staff, the gates came alive, and the enchantment began. Gate 1 activated and using a mystical Switch, the encoded input was deciphered back into its original non-negative integer form.
- Next, Binaryus spoke of the second operation, "The spell of transforming the encoded input into Hexadecimal is an ancient one. Let us invoke this charm."
 - Gate 1 was once again summoned into action, and with the grace of the Switch, the encoded input was elegantly transformed into its hexadecimal representation.
- "The third operation," Binaryus continued, "is the enchantment of swapping Gate 3 and Gate 4. A dance of bits, if you will."
 - Gate 1's energy was directed towards this mystical task, executing the spell that swapped the contents of Gate 3 and Gate 4, altering the sequence of bits in their respective gates.

- In this mystical journey through the gates and operations of binary magic, Numericus witnessed the extraordinary power encoded within these gates. The abilities of Gate 1, harnessed through the art of Switches and sorcery, were indeed a treasure trove of transformative possibilities in the digital kingdom. And thus, Numericus began to explore and utilize these enchantments, embarking on their own adventures within the vast realm of binary wonders.

Question 2

a) The base of the staircase contains 32 asterisks and height contains 16 asterisks. Each step is 6 asterisks wide and 4 asterisks high.



-
- A 15x15 grid with a triangular pattern of numbers and asterisks. The main diagonal contains numbers 1 through 15. The upper triangle contains asterisks. The lower triangle contains asterisks and numbers 1 through 14. The grid is bordered by asterisks and numbers.

Question3

A cannon ball is thrown straight into the air with a starting velocity v_0 . The position of the cannon ball after t seconds is given by the equation, $s = v_0 t - (1/2) g t^2$, where $g = 9.8 \text{ m/s}^2$ is the gravitational force.

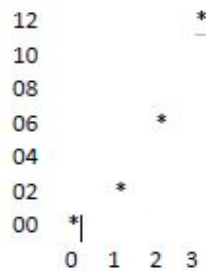
In a short time interval, the velocity v is nearly constant, and we can compute the distance the ball moves using $\Delta s = v \Delta t$. You will assume that Δt is a constant double with a value of 0.01. You can get the updated position using $s = s + v \times \Delta t$. The velocity changes constantly; in fact, it is reduced by the gravitational force of the earth. In a short time interval, $\Delta v = -g \Delta t$, you must keep the velocity updated as $v = v - g * \Delta t$; In the next iteration, the new velocity is used to update the distance. Use $\Delta t = 0.01$ for each iteration.

You are required to confirm the above given equation by a simulation. In this simulation, you will consider how the ball moves in very short time interval Δt . You need to run the simulation until the cannon ball falls back to the ground. Your program should take the initial velocity [50,150] meter/sec as an input. Update the position and velocity 100 times per second, but print out the position only every full second. Also, printout the values from the exact formula $s = v_i t - (1/2) g t^2$ for the comparison.

Lastly plot the path that cannon ball in such a way that the position of ball will get refreshed whenever you will press the spacebar. You will have to learn the usage of a command for clearing the screen.

Question 4

Write a program to plot a graph of function $f(x) = x^n + x^{n-1}$. The program shall take a maximum absolute value of x as well as a positive integer n as input. You will plot a graph for the range $[0, x]$. You should label the y-axis according to the maximum value of x . Sample Output: For $x = 3$ and $n = 2$, you should have the following output



Question 5

You are required to simulate a two-player dice rolling game using a C++ program. The program will start and ask for the Complete Names and Roll Numbers of the two players playing the game. The game has following rules.

The players roll two 6-sided dice each and get points depending on what they roll. There are 5 rounds in a game. In each round, each player rolled the dice twice.

1. The points rolled on each player's dice are added to their score.
2. If the total matches with the sum of last 2 digit of a player then the player gets additional n points where n is the last digit of the Roll Number.

3. If the total matches with m where m is index of the last alphabet of the name of the player then player gets additional 5 points. (Index of A is 1, B is 2, C is 3 and so on. Index is not case sensitive)
4. If a player rolls a double and when the sum of the roll is used as “a” in the following equation and the equation gets a single solution then the player gets an extra roll.

$$x^2 - ax + a = 0$$

5. The person with the highest score at the end of the 5 rounds wins.
6. If both players have the same score at the end of the 5 rounds, they each roll 1 die and whoever gets the highest score wins (this repeats until someone wins).

Question 6

Imagine you're strolling down a road, and there are several streetlamps along the way. First, all the lamps are turned on.

As you continue walking, you decide to do something interesting with these lamps. On your first pass, you turn off every third lamp you come across.

On the next pass, which is the second round, you revisit every second lamp and change its state - if it's off, you turn it on, and if it's on, you turn it off.

Then, you keep walking and reach the third round. This time, you toggle the state of every third lamp you encounter - again, turning it on if it's off or turning it off if it's on.

You keep repeating this pattern for n rounds, with each round toggling the lamps based on its round number. Finally, on the nth round, you only toggle the state of the last lamp you encounter.

Now, at the end of your journey, you want to know how many lamps are still glowing, or in other words, how many of them are turned on. This depends on the number of rounds you've completed and the specific pattern of toggling you've followed. The final count of lamps that are on will vary depending on the total number of lamps and the number of rounds you've walked